Time Series Analysis

with a Deep Learning Extension — Bidirectional RNN, LSTM, and GRU

What you'll learn:

1. What a time series is, and what makes it different from regular tabular data
2. The core building blocks: trend, seasonality, residuals, stationarity
3. How to diagnose a series with visualizations, decomposition, ACF/PACF
4. A classical baseline forecast (moving average) to set expectations
5. A short, intuitive introduction to **RNN → LSTM → GRU → Bidirectional RNN**
6. Solving a real forecasting problem end-to-end with LSTM/GRU/BiLSTM, and comparing results

The problem we'll solve: We are given 3 years of daily sales data for a small business. The business has an upward growth trend, yearly seasonality (busier in some months), weekly seasonality (weekday vs weekend patterns), and occasional promotional spikes. Our job is to **forecast future daily sales** using both classical and deep learning methods.

> A ready-made dataset (`daily_sales.csv`) is provided alongside this notebook so you can run every cell immediately. A cell to regenerate it from scratch is also included below in case you want to experiment with the data-generating process.

1. What Is a Time Series?

A **time series** is simply a sequence of data points collected or recorded at successive, usually equally spaced, points in time. Examples: daily stock prices, hourly temperature, monthly sales, yearly GDP, sensor readings every second.

What makes time series special compared to a normal dataset (like a table of houses and their prices) is one crucial thing: **order matters, and observations are usually not independent of each other.** Today's sales depend on yesterday's trend, this month's demand depends on the season, etc. Standard machine learning assumes rows are independent — time series analysis is largely about breaking that assumption safely.

Key concepts you must know before modeling anything

Concept	Meaning
Trend	The long-term increase or decrease in the data (e.g., a business steadily growing)
Seasonality	A pattern that repeats at a fixed, known period (daily, weekly, yearly)
Cyclic behavior	Rises and falls that are *not* of a fixed period (e.g., economic cycles)

Concept	Meaning
Noise / Residual	Random, unexplainable variation left after removing trend & seasonality
Stationarity	A series whose statistical properties (mean, variance) don't change over time
Autocorrelation	How correlated a series is with a lagged (delayed) version of itself
Lag	A past time step, e.g., "lag 1" = yesterday's value
Window / lookback	The number of past time steps used as input to predict the future

We'll meet every one of these hands-on below.

In [ ]:
```
# Basic imports we'll use throughout the notebook
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

plt.rcParams['figure.figsize'] = (12, 4)
plt.rcParams['axes.grid'] = True

np.random.seed(42)
print("Environment ready.")
```

2. The Dataset

We use a synthetic but realistic **daily sales** dataset spanning **3 years** (2022-01-01 to 2024-12-30). It was constructed by deliberately combining:

- a **linear upward trend** (the business is growing),
- **yearly seasonality** (a sine wave with a 365-day period — some months are busier),
- **weekly seasonality** (a sine wave with a 7-day period — weekday/weekend effect),
- **random Gaussian noise** (real-world unpredictability), and
- a few **promotional shock events** (sudden temporary spikes).

Because we know exactly how it was generated, it's an excellent teaching dataset: every pattern you'll learn to detect statistically is actually there by construction, so you can verify your analysis is correct.

Load the provided `daily_sales.csv` (already sitting next to this notebook):

In [ ]:
```
df = pd.read_csv("daily_sales.csv", parse_dates=["date"])
df = df.set_index("date")
print(df.shape)
df.head(10)
```

Don't have the CSV? Run the cell below to regenerate the exact same dataset from scratch (uses a fixed random seed, so it's reproducible).

In [ ]:
```
# OPTIONAL: regenerate the dataset from scratch (skip if daily_sales.csv already
def generate_sales_data():
    n_days = 3 * 365
    dates = pd.date_range(start="2022-01-01", periods=n_days, freq="D")
    t = np.arange(n_days)

    trend = 0.03 * t + 50
    yearly_seasonality = 10 * np.sin(2 * np.pi * t / 365.25)
    weekly_seasonality = 4 * np.sin(2 * np.pi * t / 7)
    noise = np.random.normal(0, 2, n_days)

    shocks = np.zeros(n_days)
    for sd in [200, 500, 800, 950]:
        shocks[sd:sd + 5] += 15  # temporary promo spike

    sales = np.round(trend + yearly_seasonality + weekly_seasonality + noise + s
    return pd.DataFrame({"date": dates, "sales": sales})

# df = generate_sales_data().set_index("date")  # uncomment to use the regenerat
```

In [ ]:
```
df.plot(y="sales", title="Daily Sales Over 3 Years", figsize=(14, 4))
plt.ylabel("Sales")
plt.show()
```

Just from this one plot we can already *see*: an overall upward slope (trend), a wavy yearly pattern, and some sharp spikes (our promo shocks). Let's zoom into a single month to see the weekly pattern, which is invisible at the 3-year zoom level.

In [ ]:
```
df.loc["2023-03-01":"2023-03-31", "sales"].plot(marker="o", title="March 2023 –
plt.ylabel("Sales")
plt.show()
```

Notice the smaller ripples every ~7 days — that's the weekly seasonality (e.g. weekend dips).

3. Decomposition: Splitting the Series into Trend + Seasonality + Residual

Time series decomposition formally separates a series y_t into:

$$y_t = T_t + S_t + R_t \quad \text{(additive model)}$$

where T_t is the trend, S_t is the seasonal component, and R_t is the residual (everything left over — ideally just noise). There's also a **multiplicative** version ($y_t = T_t \times S_t \times R_t$), used when the seasonal swings grow proportionally with the trend.

We'll use `statsmodels`' `seasonal_decompose`, a classical and very standard tool.

In [ ]:
```
from statsmodels.tsa.seasonal import seasonal_decompose

# period=365 captures the yearly cycle in daily data
decomposition = seasonal_decompose(df["sales"], model="additive", period=365)

fig = decomposition.plot()
```

```
fig.set_size_inches(12, 8)
plt.tight_layout()
plt.show()
```

Read the four panels top to bottom:

- **Observed**: the raw series (same as before)
- **Trend**: the smoothed long-term direction — should look like our steady upward slope
- **Seasonal**: the repeating yearly wave, isolated and cleaned
- **Resid**: what's left after removing trend & seasonality — should look like unstructured noise (plus a few visible spikes from our promo shocks, which the model correctly can't explain away)

This confirms our synthetic data behaves exactly as designed, and gives you the general workflow for **any** real dataset: always decompose before modeling, so you understand what you're trying to predict.

4. Stationarity: Why It Matters

Most classical time series models (like ARIMA) assume the series is **stationary**: its mean, variance, and autocorrelation structure don't change over time. Our series clearly is **not** stationary — it has a trend (mean increases over time) and seasonality (statistical properties repeat periodically).

The Augmented Dickey-Fuller (ADF) Test

The ADF test formally checks stationarity:

- **Null hypothesis (H0):** the series has a unit root → it is **non-stationary**
- If the **p-value < 0.05**, we reject H0 → the series **is stationary**

In [ ]:
```
from statsmodels.tsa.stattools import adfuller

def adf_report(series, name="series"):
    result = adfuller(series.dropna())
    print(f"ADF test for {name}")
    print(f"  ADF Statistic : {result[0]:.4f}")
    print(f"  p-value       : {result[1]:.4f}")
    print(f"  --> {'Stationary' if result[1] < 0.05 else 'NOT stationary'} (at 5
    print()

adf_report(df["sales"], "raw sales")
```

As expected, the raw series is **not stationary** (trend + seasonality inflate the p-value). The classical fix is **differencing**: subtract each value from the previous one(s) to remove trend, and subtract a full season's lag to remove seasonality.

In [ ]:
```
# First-order differencing removes the trend
diff_1 = df["sales"].diff().dropna()
```

```
# Seasonal differencing (lag=365) additionally removes the yearly seasonal patte
diff_seasonal = df["sales"].diff().diff(365).dropna()

adf_report(diff_1, "first-differenced sales")
adf_report(diff_seasonal, "first + seasonally differenced sales")

fig, axes = plt.subplots(2, 1, figsize=(12, 6), sharex=False)
diff_1.plot(ax=axes[0], title="After 1st-order differencing")
diff_seasonal.plot(ax=axes[1], title="After 1st-order + seasonal (365) differenc
plt.tight_layout()
plt.show()
```

Both transformed series now pass the stationarity test. This step is essential for classical models like ARIMA, but — spoiler for later — **deep learning models like LSTM/GRU don't strictly require stationarity**, since they can learn trend and seasonality directly from raw (or simply normalized) data. This is one of their big practical advantages.

5. Autocorrelation (ACF) and Partial Autocorrelation (PACF)

- **ACF (Autocorrelation Function):** correlation between the series and a lagged version of itself, at each lag. Tells you *how far back* memory in the series extends.
- **PACF (Partial Autocorrelation Function):** like ACF, but removes the effect of the intermediate lags — tells you the *direct* relationship at each specific lag.

These plots are the classical way to choose parameters for ARIMA-family models, and they're also a great diagnostic for **any** model: a strong spike in ACF at lag 7 confirms weekly seasonality; a spike at lag 365 confirms yearly seasonality.

In [ ]:
```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(df["sales"], lags=40, ax=axes[0])
axes[0].set_title("ACF (raw sales, first 40 lags)")
plot_pacf(df["sales"], lags=40, ax=axes[1], method="ywm")
axes[1].set_title("PACF (raw sales, first 40 lags)")
plt.tight_layout()
plt.show()
```

Notice the **wave-like pattern with peaks roughly every 7 lags** in the ACF plot — this is the fingerprint of our weekly seasonality. The slow overall decay is the fingerprint of the trend (values close in time are highly correlated).

6. A Classical Baseline: Moving Average Forecast

Before jumping to deep learning, **always build a simple baseline**. If a complex model can't beat a naive one, it's not worth the complexity. We'll use a simple moving average / naive persistence baseline and measure its error with **RMSE** and **MAE**.

We'll also set up our **train/test split** here — the *last 15% of the data* is held out as the test set, never seen during training. This mirrors the real forecasting task: predict the

future from the past.

In [ ]:
```
from sklearn.metrics import mean_squared_error, mean_absolute_error

train_size = int(len(df) * 0.85)
train_df, test_df = df.iloc[:train_size], df.iloc[train_size:]
print(f"Train size: {len(train_df)} days | Test size: {len(test_df)} days")

# Naive baseline: "tomorrow = today" (a.k.a. persistence model)
naive_pred = test_df["sales"].shift(1)
naive_pred.iloc[0] = train_df["sales"].iloc[-1]

# 7-day moving average baseline
window = 7
combined = pd.concat([train_df["sales"].tail(window), test_df["sales"]])
ma_pred = combined.rolling(window).mean().iloc[window:]

def evaluate(y_true, y_pred, name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    print(f"{name:25s} RMSE = {rmse:6.3f} | MAE = {mae:6.3f}")
    return rmse, mae

print("\n--- Baseline results on test set ---")
baseline_results = {}
baseline_results["Naive (persistence)"] = evaluate(test_df["sales"], naive_pred,
baseline_results["7-day Moving Average"] = evaluate(test_df["sales"], ma_pred, "
```

In [ ]:
```
plt.figure(figsize=(14, 4))
plt.plot(test_df.index, test_df["sales"], label="Actual", linewidth=2)
plt.plot(test_df.index, naive_pred, label="Naive (persistence)", linestyle="--")
plt.plot(test_df.index, ma_pred, label="7-day Moving Average", linestyle="--")
plt.title("Baseline Forecasts vs Actual (Test Set)")
plt.legend()
plt.show()
```

These baselines will be our **benchmark**. Any deep learning model we build should meaningfully beat these numbers, or it isn't earning its complexity.

7. A Short, Intuitive Introduction to RNN, LSTM, GRU, and Bidirectional RNNs

Classical models like moving averages or ARIMA use hand-crafted statistics (lags, differencing). **Recurrent Neural Networks (RNNs)** instead *learn* what patterns in the past are useful for predicting the future, directly from data — including nonlinear patterns that ARIMA can't capture.

7.1 Vanilla RNN — the basic idea

A standard neural network has no memory: each input is processed independently. An **RNN** adds a loop: at each time step t, it combines the current input x_t with its own previous "memory" (hidden state) h_{t-1} to produce a new hidden state h_t:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

This hidden state acts like a running summary of everything seen so far. It's simple and works for short sequences, but has a famous flaw: the **vanishing gradient problem** — when sequences get long, gradients shrink exponentially during backpropagation, so the network effectively "forgets" anything more than a few steps back.

7.2 LSTM (Long Short-Term Memory) — solving the memory problem

LSTMs (Hochreiter & Schmidhuber, 1997) fix this with a more sophisticated cell that has an explicit **cell state** (a "conveyor belt" of long-term memory) plus three **gates** that control information flow:

Gate	Question it answers
Forget gate	"What should I throw away from long-term memory?"
Input gate	"What new information should I add to long-term memory?"
Output gate	"What should I output right now, based on memory?"

Because the cell state has a near-linear path through time (gated additions/removals rather than repeated multiplications), gradients survive much longer sequences. This is why LSTMs are the classic go-to for time series and language data with long-range dependencies (e.g., remembering "this is a holiday month" from many steps back).

7.3 GRU (Gated Recurrent Unit) — a simpler, faster cousin

GRUs (Cho et al., 2014) simplify the LSTM by merging the cell state and hidden state, and using only **two gates**:

Gate	Question it answers
Update gate	"How much of the past should I keep vs. replace with new info?"
Reset gate	"How much of the past should I ignore when computing the new candidate?"

GRUs have fewer parameters than LSTMs, so they train faster and need less data, while usually performing comparably. A common rule of thumb: **try GRU first for smaller datasets or when speed matters; try LSTM when you suspect very long-range dependencies matter.**

7.4 Bidirectional RNN (Bi-RNN / BiLSTM / BiGRU)

A normal (unidirectional) RNN only reads the sequence left-to-right (past → present). A **Bidirectional RNN** runs *two* RNNs over the input — one forward, one backward — and combines (usually concatenates) their hidden states at each time step:

$$h_t = [\overrightarrow{h_t}\,;\,\overleftarrow{h_t}]$$

This lets the model use context from **both directions** when encoding each point in the input window. Bidirectionality is extremely powerful for tasks like text classification or

labeling, where the entire sequence is available at once (e.g., understanding a full sentence). **Important nuance for forecasting:** if you're predicting the *future*, your model's *final input window* only contains past data — the "future" side of a bidirectional layer applied over the input window only sees other historical days, not the actual future you're forecasting. This is still legitimate and can help the network build a richer representation of the input window, but it's not "cheating" by seeing the answer — we'll confirm this in our implementation below.

Quick comparison

	Vanilla RNN	LSTM	GRU	Bidirectional (any of the above)
Memory	Short only	Long + short	Long + short	Same as base cell, richer context
Gates	0	3	2	—
Speed	Fastest	Slower	Faster than LSTM	~2x base cell (two passes)
Good for	Very short sequences	Long dependencies	Similar to LSTM, less data/compute	Encoding a fixed window fully, not causal generation

In [ ]:
```
# A tiny illustrative sketch of the gate equations is worth seeing once as code,
# This is NOT how you'd actually implement Keras layers (Keras does this interna
# it's just to make the LSTM gate equations concrete.

def lstm_cell_step(x_t, h_prev, c_prev, W_f, W_i, W_o, W_c):
    sigmoid = lambda z: 1 / (1 + np.exp(-z))
    concat = np.concatenate([h_prev, x_t])

    f_t = sigmoid(W_f @ concat)          # forget gate: what to erase from memor
    i_t = sigmoid(W_i @ concat)          # input gate: what new info to add
    o_t = sigmoid(W_o @ concat)          # output gate: what to reveal as output
    c_hat_t = np.tanh(W_c @ concat)      # candidate new memory content

    c_t = f_t * c_prev + i_t * c_hat_t   # update long-term memory (cell state)
    h_t = o_t * np.tanh(c_t)             # compute new hidden state (short-term
    return h_t, c_t

print("This is the literal equation an LSTM cell runs at every single time step.
print("Keras implements this efficiently in compiled code, and updates the weigh
print("matrices (W_f, W_i, W_o, W_c) via backpropagation through time during tra
```

8. Preparing the Data for Sequence Models

Neural networks expect fixed-size numeric inputs, so we must convert our single long series into **(input window → next value)** pairs using a **sliding window**. For example, with a 30-day window: use days 1-30 to predict day 31, then days 2-31 to predict day 32, etc.

We also **scale the data to [0, 1]** with `MinMaxScaler`. This is important for RNN-family models: it keeps values in a numeric range where `tanh` / `sigmoid` activations behave well and speeds up convergence. Note we fit the scaler **only on the training set**, then apply it to the test set — fitting on the whole dataset would leak future information into training (a common and important mistake to avoid).

In [ ]:
```
from sklearn.preprocessing import MinMaxScaler

WINDOW = 30  # use the last 30 days to predict the next day

series_values = df["sales"].values.reshape(-1, 1)

train_size = int(len(series_values) * 0.85)
train_raw, test_raw = series_values[:train_size], series_values[train_size:]

scaler = MinMaxScaler()
train_scaled = scaler.fit_transform(train_raw)          # fit ONLY on train
test_scaled = scaler.transform(test_raw)                # reuse train's scaling

def make_windows(data, window):
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[i:i + window, 0])
        y.append(data[i + window, 0])
    return np.array(X), np.array(y)

X_train, y_train = make_windows(train_scaled, WINDOW)
X_test, y_test = make_windows(test_scaled, WINDOW)

# Reshape to (samples, timesteps, features) — the shape Keras RNN layers expect
X_train = X_train.reshape((X_train.shape[0], WINDOW, 1))
X_test = X_test.reshape((X_test.shape[0], WINDOW, 1))

print("X_train:", X_train.shape, " y_train:", y_train.shape)
print("X_test :", X_test.shape, " y_test :", y_test.shape)
```

9. Building and Training LSTM, GRU, and Bidirectional LSTM Models

We'll build three small models with a nearly identical architecture so the comparison is fair — only the recurrent layer type changes:

1. **LSTM** — the classic long-memory cell
2. **GRU** — the lighter, faster alternative
3. **Bidirectional LSTM** — reads the input window in both directions for a richer encoding

Each model: one recurrent layer (32 units) → Dense(1) output layer, trained with the Adam optimizer and Mean Squared Error loss (standard choice for regression).

In [ ]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

```
tf.random.set_seed(42)

def build_model(cell_type="LSTM"):
    inputs = keras.Input(shape=(WINDOW, 1))
    if cell_type == "LSTM":
        x = layers.LSTM(32)(inputs)
    elif cell_type == "GRU":
        x = layers.GRU(32)(inputs)
    elif cell_type == "BiLSTM":
        x = layers.Bidirectional(layers.LSTM(32))(inputs)
    else:
        raise ValueError("Unknown cell_type")
    outputs = layers.Dense(1)(x)
    model = keras.Model(inputs, outputs)
    model.compile(optimizer="adam", loss="mse")
    return model

models = {
    "LSTM": build_model("LSTM"),
    "GRU": build_model("GRU"),
    "BiLSTM": build_model("BiLSTM"),
}

models["LSTM"].summary()
```

In [ ]:
```
EPOCHS = 25
BATCH_SIZE = 16

histories = {}
early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, resto

for name, model in models.items():
    print(f"\n--- Training {name} ---")
    hist = model.fit(
        X_train, y_train,
        validation_split=0.1,
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        callbacks=[early_stop],
        verbose=0,
    )
    histories[name] = hist
    print(f"{name}: final train loss = {hist.history['loss'][-1]:.5f}, "
          f"final val loss = {hist.history['val_loss'][-1]:.5f}, "
          f"epochs run = {len(hist.history['loss'])}")
```

In [ ]:
```
plt.figure(figsize=(10, 4))
for name, hist in histories.items():
    plt.plot(hist.history["val_loss"], label=f"{name} val_loss")
plt.title("Validation Loss During Training")
plt.xlabel("Epoch")
plt.ylabel("MSE (scaled)")
plt.legend()
plt.show()
```

10. Evaluating and Comparing All Models

We predict on the held-out test set, **inverse-transform** predictions back to the original sales scale, and compute RMSE/MAE — the same metrics we used for our classical baselines, so we can directly compare deep learning vs. classical performance.

In [ ]:
```
results = dict(baseline_results)  # start with the classical baselines from Sect
predictions = {}

for name, model in models.items():
    pred_scaled = model.predict(X_test, verbose=0)
    pred = scaler.inverse_transform(pred_scaled)
    true = scaler.inverse_transform(y_test.reshape(-1, 1))
    predictions[name] = pred.flatten()
    results[name] = evaluate(true.flatten(), pred.flatten(), name)

print("\n--- Full comparison (lower is better) ---")
comparison_df = pd.DataFrame(results, index=["RMSE", "MAE"]).T.sort_values("RMSE
comparison_df
```

In [ ]:
```
# Align the test dates with our windowed predictions (first WINDOW days of test
# are "consumed" as input context and have no prediction)
test_dates = test_df.index[WINDOW:]
actual_values = scaler.inverse_transform(y_test.reshape(-1, 1)).flatten()

plt.figure(figsize=(14, 5))
plt.plot(test_dates, actual_values, label="Actual", linewidth=2.5, color="black"
for name, pred in predictions.items():
    plt.plot(test_dates, pred, label=name, linestyle="--")
plt.title("Deep Learning Forecasts vs Actual Sales (Test Set)")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.show()
```

How to read the results

- If **LSTM/GRU/BiLSTM all beat both classical baselines**, it confirms the deep models are successfully learning the nonlinear combination of trend + seasonality + shocks we built into the data.
- **GRU vs. LSTM** performance is often close on data like this — GRU converges a bit faster since it has fewer parameters, while LSTM sometimes eventually edges ahead with more data or longer sequences.
- **BiLSTM** typically produces the smoothest, most context-aware predictions, because each point in its 30-day window was encoded using information from *both* directions of that window — but remember, it still only ever sees the past 30 days when forecasting the true future; the "future" it looks at internally is other historical days inside the input window, not the day being predicted.
- If deep learning **doesn't** clearly beat the moving-average baseline here, that's a realistic and important lesson too: with smooth, strongly seasonal data and limited size, simple methods are often "good enough," and the extra complexity of RNNs pays off more on noisier, longer, or more irregular real-world series.

11. Forecasting Genuinely Unseen Future Days

As a final practical step, let's use our best model to forecast **14 brand-new days** beyond the end of our dataset — something with no "actual" value to compare against yet, exactly like a real production forecasting task. We do this by **recursively feeding each prediction back in** as if it were an observed value, then sliding the window forward.

In [ ]:
```
best_model_name = comparison_df.index[0]
best_model = models[best_model_name] if best_model_name in models else None

if best_model is None:
    print(f"Best model by RMSE was a classical baseline ({best_model_name}); "
          f"using BiLSTM for the recursive forecast demo instead.")
    best_model_name = "BiLSTM"
    best_model = models["BiLSTM"]

print(f"Using {best_model_name} to forecast 14 future days...")

FUTURE_STEPS = 14
last_window = series_values[-WINDOW:].reshape(-1, 1)
last_window_scaled = scaler.transform(last_window).flatten().tolist()

future_preds_scaled = []
window_buffer = last_window_scaled.copy()

for _ in range(FUTURE_STEPS):
    x_input = np.array(window_buffer[-WINDOW:]).reshape(1, WINDOW, 1)
    next_scaled = best_model.predict(x_input, verbose=0)[0, 0]
    future_preds_scaled.append(next_scaled)
    window_buffer.append(next_scaled)

future_preds = scaler.inverse_transform(np.array(future_preds_scaled).reshape(-1
future_dates = pd.date_range(start=df.index[-1] + pd.Timedelta(days=1), periods=

forecast_df = pd.DataFrame({"date": future_dates, f"{best_model_name}_forecast":
forecast_df
```

In [ ]:
```
plt.figure(figsize=(14, 5))
plt.plot(df.index[-90:], df["sales"].iloc[-90:], label="Recent actual sales", co
plt.plot(future_dates, future_preds, label=f"{best_model_name} 14-day forecast",
         color="tab:red", marker="o")
plt.axvline(df.index[-1], color="gray", linestyle=":", label="Forecast start")
plt.title(f"14-Day Future Forecast using {best_model_name}")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.show()
```

A note on recursive forecasting: each new prediction is fed back in as if it were real, so small errors can compound the further out you forecast — this is why multi-step forecasts are inherently harder and less certain than single-step-ahead forecasts. In practice, always widen your uncertainty expectations the further into the future you go.

12. Summary & Next Steps

What we covered:

- Time series fundamentals: trend, seasonality, stationarity, ACF/PACF
- Classical decomposition and a simple moving-average baseline
- Intuitive theory of RNN → LSTM → GRU → Bidirectional RNNs, and *why* each innovation exists
- Built and trained LSTM, GRU, and BiLSTM models on real windowed sequence data
- Compared all models against classical baselines using RMSE/MAE
- Produced a genuine multi-step future forecast

Where to go next:

- Try a longer/shorter `WINDOW` size and see how it affects accuracy
- Add more features beyond raw sales (e.g., day-of-week, month, is_holiday) as extra input channels
- Try stacking recurrent layers (e.g., two LSTM layers) or adding Dropout for regularization